

U-Net Architecture

Source: `task2_predict_rings_ml/ml/model.py`

All figures below (shapes, parameter counts) were verified by directly reading and running this file, not estimated from memory.

1. Overview

This model takes a color photograph of a clam shell cross-section (letterboxed to a 512x512 square) and outputs a same-resolution map of per-pixel scores: how likely each pixel is to lie on the true growth-band centerline. It is a U-Net -- an encoder-decoder convolutional network with skip connections, a design originally introduced by Ronneberger, Fischer & Brox (2015) for biomedical image segmentation, and now used extremely broadly for any task requiring a per-pixel prediction aligned to the input image.

The encoder repeatedly shrinks the image spatially while increasing the number of learned feature channels, trading spatial resolution for a richer feature representation at each scale. The decoder mirrors this in reverse, growing the spatial size back to the original resolution. At each decoder step, it is given a copy of the encoder's features from the matching resolution (a skip connection), so fine spatial detail lost during downsampling does not have to be reconstructed from a small, low-resolution bottleneck alone.

Provenance note: this implementation was written from general knowledge of the standard U-Net pattern, not copied from a specific paper's source code or a specific tutorial. It is a commonly used simplified variant -- same-padding convolutions and batch normalization -- rather than a reproduction of the original 2015 paper's exact specification (which used unpadded convolutions with cropping, and predates the widespread use of batch norm).

2. Input / Output Contract

- Input: tensor of shape (N, 3, 512, 512) -- N images, 3 RGB channels, 512x512 pixels.
- Output: tensor of shape (N, 1, 512, 512) -- one raw logit per pixel.

The output is a raw, unbounded logit, not a probability -- no sigmoid is applied inside the model. A sigmoid is applied afterward: implicitly inside the training loss function (nn.BCEWithLogitsLoss applies it internally for numerical stability), or explicitly during inference (torch.sigmoid() is called on the output in the separate inference code).

3. The Repeated Building Block: conv_block

Every stage of the network, encoder and decoder alike, is built from the same function, which returns two convolution layers in sequence:

```
def conv_block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(inplace=True),
        nn.Conv2d(cout, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(inplace=True),
    )
```

- Conv2d(cin, cout, 3, padding=1): a 3x3 convolution, cin input channels to cout output channels. padding=1 keeps the output the same height/width as the input (a 3x3 kernel with no padding would shrink each side by 1 pixel).
- BatchNorm2d(cout): normalizes each channel's activations across the batch, then applies a learned per-channel scale and shift. Stabilizes and speeds up training.
- ReLU(inplace=True): the activation $\max(0, x)$. Without a non-linearity, stacking convolutions would mathematically collapse into one linear operation regardless of depth. inplace=True overwrites the input tensor's memory rather than allocating a new one -- a memory optimization with no effect on the output values.

Two conv
representa

4. Full Architecture

Four downsampling levels, a bottleneck, and four upsampling levels with skip connections. Channel width is controlled by one parameter, `base=16`.

4.1 Encoder (contracting path)

```
self.enc1 = conv_block(3, base)          # base = 16
self.enc2 = conv_block(base, base * 2)
self.enc3 = conv_block(base * 2, base * 4)
self.enc4 = conv_block(base * 4, base * 8)
self.pool = nn.MaxPool2d(2)
```

Each encoder block doubles the channel count (16, 32, 64, 128) -- the standard convention of doubling channels each time spatial resolution is halved. `self.pool` is a single 2x2 max-pooling layer (no learnable parameters), reused at every downsampling step: it keeps the maximum value in each non-overlapping 2x2 pixel block, halving height and width.

4.2 Bottleneck

```
self.bottleneck = conv_block(base * 8, base * 16)
```

The bottom of the U: one more block, 128 to 256 channels, at the smallest spatial resolution -- the most channels, least spatial detail, most abstract representation in the network.

4.3 Decoder (expanding path)

```
self.up4 = nn.ConvTranspose2d(base * 16, base * 8, 2, stride=2)
self.dec4 = conv_block(base * 16, base * 8)
self.up3 = nn.ConvTranspose2d(base * 8, base * 4, 2, stride=2)
self.dec3 = conv_block(base * 8, base * 4)
self.up2 = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
self.dec2 = conv_block(base * 4, base * 2)
self.up1 = nn.ConvTranspose2d(base * 2, base, 2, stride=2)
self.dec1 = conv_block(base * 2, base)
```

Each up-step has two parts. First, a `ConvTranspose2d` (kernel 2, stride 2): a transposed convolution that doubles spatial size and halves channel count. Unlike simple nearest- or bilinear-upsampling, this has learnable weights -- the network learns how to upsample rather than following a fixed rule. Second, the `conv_block`'s declared input channel count is double what the upsample alone would produce (e.g. `dec4` takes `base*16=256` in, not `base*8=128`) -- because the decoder concatenates the upsampled features with the matching-resolution encoder output (the skip connection) before the block runs, doubling the channel count going in.

4.4 Output layer

```
self.out = nn.Conv2d(base, 1, 1)
```

A final 1x1 convolution, 16 channels to 1. A 1x1 kernel looks at no neighboring pixels -- it is a per-pixel linear combination of that pixel's 16 feature values, collapsing them into the single output logit described in Section 2.

5. Data Flow and Verified Tensor Shapes

The forward() method below is the actual, complete implementation (model.py, lines 38-49). Shapes shown are for a batch size of 1 with a 512x512 input -- the size actually used throughout this project -- and were confirmed by running the model, not calculated by hand.

```
def forward(self, x):
    e1 = self.enc1(x)
    e2 = self.enc2(self.pool(e1))
    e3 = self.enc3(self.pool(e2))
    e4 = self.enc4(self.pool(e3))
    b = self.bottleneck(self.pool(e4))

    d4 = self.dec4(torch.cat([self.up4(b), e4], dim=1))
    d3 = self.dec3(torch.cat([self.up3(d4), e3], dim=1))
    d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
    d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
    return self.out(d1)
```

Stage	Operation	Output shape (N=1)
Input	--	(1, 3, 512, 512)
enc1	conv_block(3->16)	(1, 16, 512, 512)
	maxpool 2x2	(1, 16, 256, 256)
enc2	conv_block(16->32)	(1, 32, 256, 256)
	maxpool 2x2	(1, 32, 128, 128)
enc3	conv_block(32->64)	(1, 64, 128, 128)
	maxpool 2x2	(1, 64, 64, 64)
enc4	conv_block(64->128)	(1, 128, 64, 64)
	maxpool 2x2	(1, 128, 32, 32)
bottleneck	conv_block(128->256)	(1, 256, 32, 32)
up4+concat(e4)	transposed conv + skip	(1, 256, 64, 64)
dec4	conv_block(256->128)	(1, 128, 64, 64)
up3+concat(e3)	transposed conv + skip	(1, 128, 128, 128)
dec3	conv_block(128->64)	(1, 64, 128, 128)
up2+concat(e2)	transposed conv + skip	(1, 64, 256, 256)
dec2	conv_block(64->32)	(1, 32, 256, 256)
up1+concat(e1)	transposed conv + skip	(1, 32, 512, 512)
dec1	conv_block(32->16)	(1, 16, 512, 512)
out	1x1 conv (16->1)	(1, 1, 512, 512)

6. Parameter Count (verified by execution)

Computed by instantiating the model and summing `p.numel()` for every parameter tensor -- an exact count, not an estimate.

Component	Parameters
enc1	2,832
enc2	14,016
enc3	55,680
enc4	221,952
bottleneck	886,272
up4	131,200
dec4	443,136
up3	32,832
dec3	110,976
up2	8,224
dec2	27,840
up1	2,064
dec1	7,008
out	17
TOTAL	1,944,049

All ~1.94 million parameters are trainable; none are frozen or pretrained -- the model is trained entirely from scratch (see `model_provenance` section below). As a cross-check: the saved checkpoint file is 7,830,233 bytes, and 1,944,049 parameters x 4 bytes/float32 is approximately 7.78 MB, consistent with a plain float32 state dict, which is what `torch.save(model.state_dict(), ...)` produces.

For scale reference: this is a small model. A full-size U-Net on natural images often runs tens of millions of parameters; a ResNet-50 backbone alone is roughly 25 million.

7. Design Choices and Their Justification

Being direct about which choices are principled versus which are reasonable but untested judgment calls:

- base=16, 4 levels: chosen because the training set is small (32 to 107 images over the course of this project). A wider/deeper network has more parameters to overfit with, and is also slower to train on CPU -- the only hardware available in this environment. This was NOT tuned via systematic search; it is a reasonable starting guess, not a validated optimum.
- 4 downsampling levels: $512 / 2^4 = 32$, a reasonably small but non-degenerate bottleneck. Not derived from a more principled calculation.
- 3x3 kernels with same-padding: the near-universal default in modern convolutional networks since VGG-style architectures -- a stack of 3x3 convolutions approximates a larger receptive field with fewer parameters than one large kernel.
- ConvTranspose2d for upsampling (rather than fixed interpolation + convolution): a design choice,

not benchmarked against the alternative in this project. Some practitioners prefer plain upsampling because transposed convolutions can produce checkerboard artifacts; both options were not compared here.

- No activation on the final output: deliberate, so raw logits can be passed directly to a numerically-stable combined sigmoid+loss function during training.

8. Scope of This Document

This document covers only the architecture defined in `model.py`. It does not cover: how the model is trained (loss function, optimizer, data augmentation, epochs -- defined in a separate training script), how its output is converted into a final centerline (confidence-weighted extraction and gap-bridging logic in separate inference code), what data it was trained on, or how that data was collected. Those are documented separately.